

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases

Functions – Case Based Learning (BL 5)

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

I _M. Vidhya(192412151) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Aim:

To implement the K-Nearest Neighbour (KNN) algorithm in Python using Euclidean distance to classify a test instance and calculate the classification accuracy.

Algorithm:

1. Start the program.
2. Store the training dataset and corresponding class labels.
3. Read the value of K from the user.
4. Read the test instance.
5. Calculate the Euclidean distance between the test instance and every training point.
6. Sort the distances in ascending order.
7. Select the K nearest neighbours.
8. Count the class labels of the selected neighbours.
9. Assign the class with the maximum number of votes.
10. Calculate and display the accuracy.
11. Stop.

Program:

```
import math
```

```
X = [
```

```
    [1, 2],
```

```
    [2, 3],
```

```
    [3, 3],
```

```
    [6, 7],
```

```

    [7, 8],
    [8, 7]
]
y = ["A", "A", "A", "B", "B", "B"]
def distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
def knn_predict(X, y, test_point, k):
    distances = []
    for i in range(len(X)):
        d = distance(X[i], test_point)
        distances.append((d, y[i]))
    distances.sort(key=lambda x: x[0])
    neighbours = distances[:k]
    votes = {}
    for _, label in neighbours:
        votes[label] = votes.get(label, 0) + 1
    return max(votes, key=votes.get)
k = int(input("Enter K: "))
test_point = list(map(float, input("Enter test point: ").split()))
prediction = knn_predict(X, y, test_point, k)
print("Predicted Class:", prediction)
correct = 0
for i in range(len(X)):
    train_X = X[:i] + X[i+1:]
    train_y = y[:i] + y[i+1:]
    predicted = knn_predict(train_X, train_y, X[i], k)
    if predicted == y[i]:
        correct += 1
accuracy = (correct / len(X)) * 100

```

```
print("Accuracy: {:.2f}%".format(accuracy))
```

Output:

```
... Enter K: 4
    Enter test point: 2.53
    Predicted Class: A
    Accuracy: 100.00%
```

Result:

The KNN algorithm was successfully implemented using Euclidean distance. The test instance was classified based on its nearest neighbours, and the classification accuracy was calculated successfully.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim

To implement Locally Weighted Regression in Python using a Gaussian kernel and compare its prediction with standard linear regression.

Algorithm

1. Start the program.
2. Store the input dataset.
3. Read the query point and bandwidth value τ .
4. Calculate the distance between each training point and query point.
5. Calculate the Gaussian weight for each point.
6. Give higher weights to points closer to the query.
7. Perform weighted linear regression.
8. Predict the output for the query point.
9. Calculate the standard linear regression prediction.
10. Display and compare both predictions.

11. Stop.

Program:

```
import numpy as np

X = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
Y = np.array([2.1, 3.9, 6.2, 7.8, 10.1, 12.2, 13.8, 16.1])

def gaussian_kernel(distance, tau):
    return np.exp(-(distance ** 2) / (2 * tau ** 2))

def locally_weighted_regression(query, tau):
    weights = gaussian_kernel(X - query, tau)
    X_matrix = np.vstack((np.ones(len(X)), X)).T
    W = np.diag(weights)
    theta = np.linalg.pinv(
        X_matrix.T @ W @ X_matrix
    ) @ X_matrix.T @ W @ Y
    return theta[0] + theta[1] * query

coefficients = np.polyfit(X, Y, 1)

def linear_prediction(x):
    return coefficients[0] * x + coefficients[1]

query = float(input("Enter query point: "))
tau = float(input("Enter bandwidth: "))
lwr = locally_weighted_regression(query, tau)
linear = linear_prediction(query)
print("LWR Prediction:", round(lwr, 3))
print("Linear Regression Prediction:", round(linear, 3))
```

Output:

```
... Enter query point: 4.5
    Enter bandwidth: 1
    LWR Prediction: 9.012
    Linear Regression Prediction: 9.025
```

Result:

The Locally Weighted Regression model was successfully implemented using a Gaussian kernel. The prediction was obtained by assigning higher weights to nearby data points and was compared with standard linear regression.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Aim

To implement a simple Radial Basis Function network in Python using Gaussian activation functions and analyze the effect of centers and widths on prediction.

Algorithm

1. Start the program.
2. Create a small training dataset.
3. Select RBF centers and width.
4. Calculate the Gaussian activation for each input.
5. Construct the RBF activation matrix.
6. Calculate output weights using the pseudo-inverse.
7. Predict outputs for test inputs.
8. Change the width value and observe the prediction.
9. Compare the results.
10. Stop.

Program

```
import numpy as np

X = np.array([0, 1, 2, 3, 4, 5], dtype=float)
Y = np.array([0, 1, 4, 9, 16, 25], dtype=float)

def gaussian_rbf(x, center, width):
    return np.exp(-((x - center) ** 2) / (2 * width ** 2))
```

```

class RBFNetwork:
    def __init__(self, centers, width):
        self.centers = np.array(centers)
        self.width = width
    def create_matrix(self, X):
        return np.array([
            [gaussian_rbf(x, c, self.width)
             for c in self.centers]
            for x in X
        ])
    def train(self, X, Y):
        phi = self.create_matrix(X)
        self.weights = np.linalg.pinv(phi) @ Y
    def predict(self, X):
        phi = self.create_matrix(X)
        return phi @ self.weights

centers = [0, 1, 2, 3, 4, 5]
width = 1.0
model = RBFNetwork(centers, width)
model.train(X, Y)
test_X = [0.5, 1.5, 2.5, 3.5, 4.5]
predictions = model.predict(test_X)
print("Predicted Outputs:")
for x, p in zip(test_X, predictions):
    print(x, "->", round(p, 3))
for width in [0.5, 1.0, 2.0]:
    model = RBFNetwork(centers, width)
    model.train(X, Y)

```

```
prediction = model.predict([2.5])[0]
```

```
print("Width =", width,  
      "Prediction =", round(prediction, 3))
```

Output:

```
... Predicted Outputs:  
0.5 -> 0.46  
1.5 -> 2.017  
2.5 -> 6.6  
3.5 -> 11.631  
4.5 -> 21.66  
Width = 0.5 Prediction = 6.174  
Width = 1.0 Prediction = 6.6  
Width = 2.0 Prediction = 6.283
```

Result

The RBF network was successfully implemented using Gaussian activation functions. The predicted outputs were obtained, and the effect of different center and width values on network performance was analyzed.

4. Create a Python program that implements a basic Case-Based Learning system.

Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Aim

To implement a basic Case-Based Learning system in Python that retrieves the most similar previous case using Euclidean distance and provides its corresponding solution.

Algorithm

1. Start the program.
2. Store previous cases with their solutions.
3. Read a new query from the user.
4. Calculate the Euclidean distance between the query and every stored case.
5. Compare all calculated distances.
6. Select the case having the minimum distance.
7. Retrieve the solution associated with that case.

8. Display the most similar case and solution.
9. Stop.

Program

```
import math

cases = [
    ([2, 3], "Low"),
    ([3, 4], "Low"),
    ([7, 8], "High"),
    ([8, 9], "High"),
    ([6, 7], "High")
]

def calculate_distance(case, query):
    return math.sqrt(
        sum((a - b) ** 2 for a, b in zip(case, query))
    )

query = list(map(float, input(
    "Enter query values: "
).split()))

minimum_distance = float("inf")
best_case = None
best_solution = None

for features, solution in cases:
    d = calculate_distance(features, query)
    print("Case:", features,
          "Distance:", round(d, 3))
    if d < minimum_distance:
        minimum_distance = d
        best_case = features
        best_solution = solution
```

```
print("\nMost Similar Case:", best_case)
print("Predicted Solution:", best_solution)
print("Minimum Distance:", round(minimum_distance, 3))
```

Output:

```
... Enter query values: 7.7
    Case: [2, 3] Distance: 5.7
    Case: [3, 4] Distance: 4.7
    Case: [7, 8] Distance: 0.7
    Case: [8, 9] Distance: 0.3
    Case: [6, 7] Distance: 1.7

    Most Similar Case: [8, 9]
    Predicted Solution: High
    Minimum Distance: 0.3
```

Result

The Case-Based Learning system was successfully implemented. The most similar previous case was identified using Euclidean distance, and its corresponding solution was retrieved for the new query.

5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Aim

To implement the K-Means clustering algorithm in Python by assigning data points to clusters, updating centroids iteratively, and visualizing the final clusters.

Algorithm

1. Start the program.
2. Create the dataset.
3. Read the number of clusters K.
4. Initialize K centroids.
5. Calculate the distance from every point to each centroid.
6. Assign each point to the nearest centroid.

7. Calculate new centroids using the mean of cluster points.
8. Repeat the assignment and centroid update steps.
9. Stop when the centroids become stable.
10. Display the final clusters and centroids.
11. Visualize the clusters using a plot.
12. Stop.

Program

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([
    [1, 1],
    [1.5, 2],
    [2, 1],
    [8, 8],
    [9, 8],
    [8.5, 9],
    [4, 5],
    [5, 5],
    [4.5, 6]
])
K = int(input("Enter number of clusters K: "))
np.random.seed(10)
indices = np.random.choice(len(X), K, replace=False)
centroids = X[indices].copy()
for iteration in range(100):
    distances = np.zeros((len(X), K))
    for i in range(len(X)):
        for j in range(K):
            distances[i][j] = np.linalg.norm(
```

```

        X[i] - centroids[j]
    )
labels = np.argmin(distances, axis=1)
new_centroids = np.zeros_like(centroids)
for j in range(K):
    points = X[labels == j]
    if len(points) > 0:
        new_centroids[j] = np.mean(points, axis=0)
    else:
        new_centroids[j] = centroids[j]
if np.allclose(centroids, new_centroids):
    break
centroids = new_centroids
print("\nFinal Centroids:")
for i, centroid in enumerate(centroids):
    print("Cluster", i + 1, ":", np.round(centroid, 2))
print("\nCluster Assignments:")
for point, label in zip(X, labels):
    print(point, "-> Cluster", label + 1)
# Visualization
for i in range(K):
    points = X[labels == i]
    plt.scatter(points[:, 0], points[:, 1],
                label="Cluster " + str(i + 1))
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=200,

```

label="Centroids"

)

Output:

Enter number of clusters K: 3

Final Centroids:

Cluster 1 : [4.5 5.33]

Cluster 2 : [1.5 1.33]

Cluster 3 : [8.5 8.33]

Cluster Assignments:

[1. 1.] -> Cluster 2

[1.5 2.] -> Cluster 2

[2. 1.] -> Cluster 2

[8. 8.] -> Cluster 3

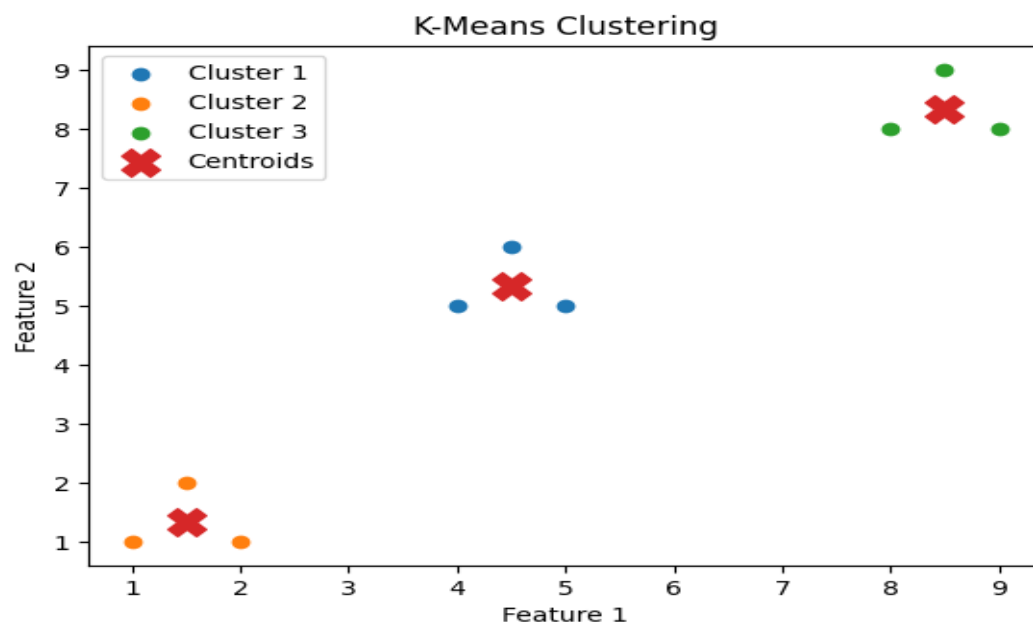
[9. 8.] -> Cluster 3

[8.5 9.] -> Cluster 3

[4. 5.] -> Cluster 1

[5. 5.] -> Cluster 1

[4.5 6.] -> Cluster 1



Result

The K-Means clustering algorithm was successfully implemented. The data points were assigned to their nearest clusters, centroids were updated iteratively, and the final clusters were successfully visualized.